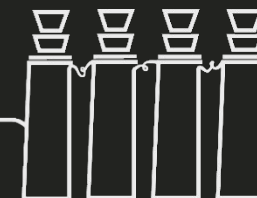


TRABALHO PRÁTICO 01 LISTA ENCADEADA

Algoritmos e Estruturas de Dados I – CCF 211

Gabriel Henrique R. Macedo – 07237

Diego Soares Carvalho – 07225

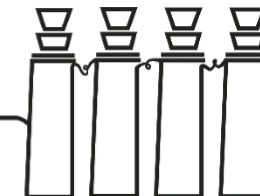


O problema

- Uma explosão na AEDS fez os Pokémon fugirem do Centro de Pesquisa, e dois treinadores saem para resgatá-los.
- O programa lê os treinadores e os fugitivos de um arquivo, conduz a missão e mostra cada passo no terminal.
- No fim, o Centro grava um relatório em .txt com os Pokémon recuperados.
- Exigências: as listas são **obrigatoriamente encadeadas** e tudo é TAD: Pokémon, PokeLista, Treinador e Centro de Pesquisa.

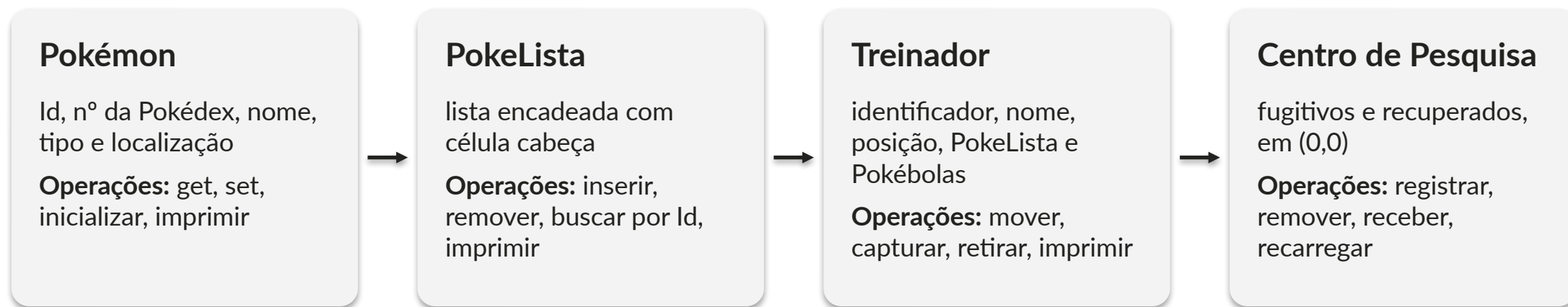
Regras da missão

- 1 Vai o treinador **mais perto** do Pokémon (distância euclidiana). Empate: o de **menor identificador**.
- 2 Cada captura gasta **uma Pokébola**, e o Centro é avisado na hora.
- 3 Sem Pokébolas: volta ao Centro, entrega o que pegou e recebe de **1 a 20**.
- 4 Último resgate: os **dois voltam** e entregam tudo.

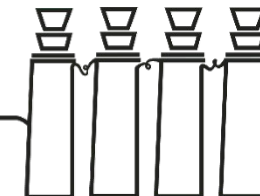


Modelagem do problema

- Começamos pelos TADs, **de baixo para cima**: cada um usa só o anterior, então não existe inclusão circular.
- O controle da missão ficou num **módulo separado** (`missao.c`); o main só prepara o sorteio e chama a missão.
- **Divisão de tarefas**: escrevemos o código juntos, um digitando e o outro revisando, alternando a cada módulo. Assim os dois sabem explicar o projeto inteiro.



Os quatro TADs pedidos: cada seta indica que o da direita usa o da esquerda



Propostas e desafios

1 O Id do Pokémon

Pensamos em usar o número da Pokédex, mas ele repete: o teste2.txt traz quatro Pikachus 025. O Id virou a **ordem de leitura** (1, 2, 3...).

2 A comparação de distâncias

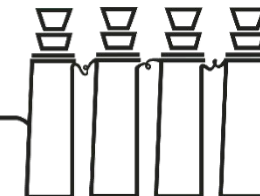
Em vez de comparar raízes, comparamos o **quadrado** da distância. A ordem é a mesma, o empate fica exato e a raiz só aparece na impressão.

3 A remoção na lista

O desafio foi o apontador ultimo: se o removido era o último, ele precisa voltar para o anterior **antes do free**, senão aponta para memória liberada.

4 Uma versão mais simples

Começamos com menu e muitas validações. Na revisão, enxugamos para o que o enunciado pede: o programa pede o caminho do arquivo e roda a missão.



Estrutura do projeto

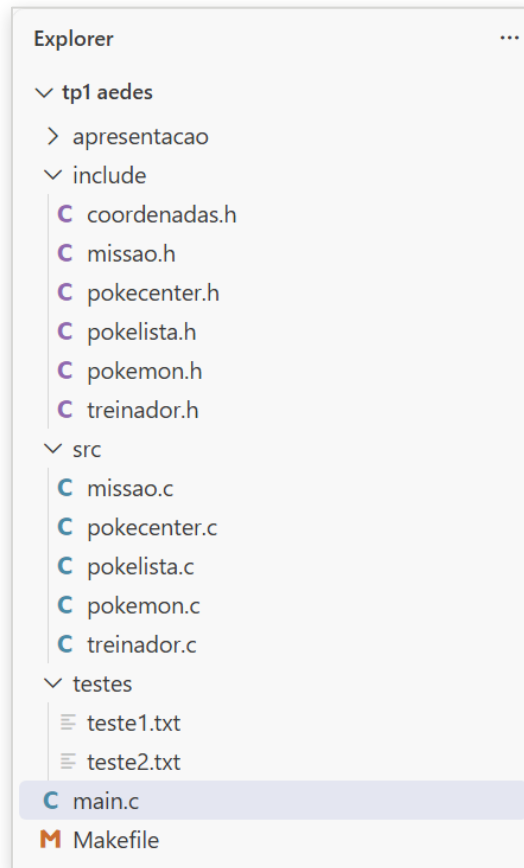
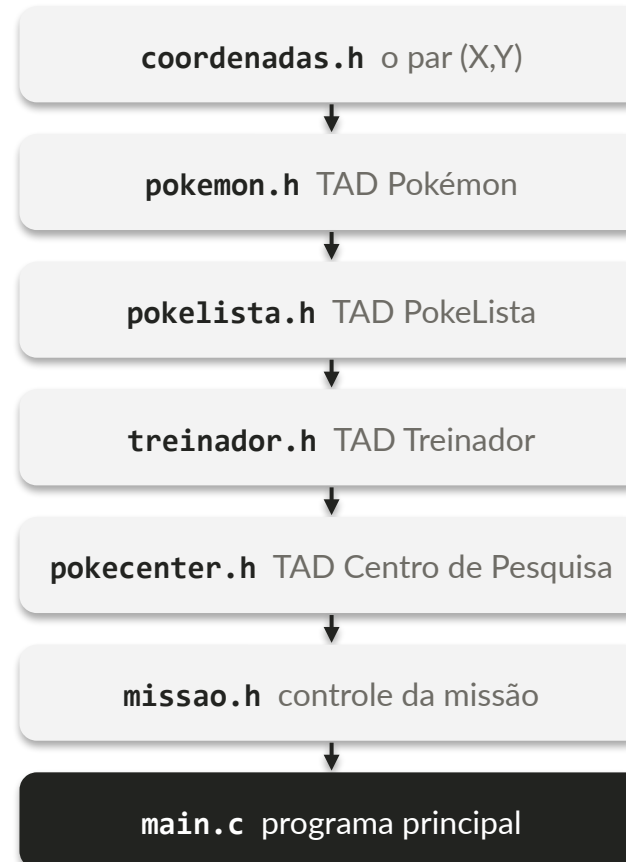
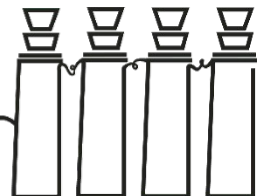


Figura 01 – O projeto no VS Code



cada arquivo inclui o de cima

- include/ tem os seis .h e src/ os cinco .c. Cada .c inclui só o seu .h.
- Os .h formam uma cadeia em linha reta: não há inclusão circular.
- O Makefile compila tudo com -Wall -Wextra -std=c99: mingw32-make no Windows, make no Linux.
- testes/ traz os dois arquivos oficiais do Moodle.



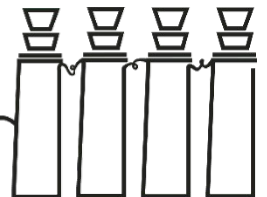
```
C pokemon.h include X
8 // tamanho dos vetores
9 #define TAM_NOME 30
10 #define TAM_TIPO 20
11
12 typedef struct {
13     int identificacao;
14     int numPokedex;
15     char nome[TAM_NOME];
16     char tipo[TAM_TIPO];
17     Coordenada localizacao;
18 } Pokemon;
```

Figura 02 – A struct, em include/pokemon.h

```
C pokemon.c src X
6 void pokemonInicializar(Pokemon *p, int identificacao, int
  numPokedex, const char *nome, const char *tipo, int cordX, int
  cordY)
7 {
8     pokemonSetId(p, identificacao);
9     pokemonSetNumPokedex(p, numPokedex);
10    pokemonSetName(p, nome);
11    pokemonSetTipo(p, tipo);
12    pokemonSetLocalizacao(p, cordX, cordY);
13 }
```

Figura 03 – A inicialização chama os set, em src/pokemon.c

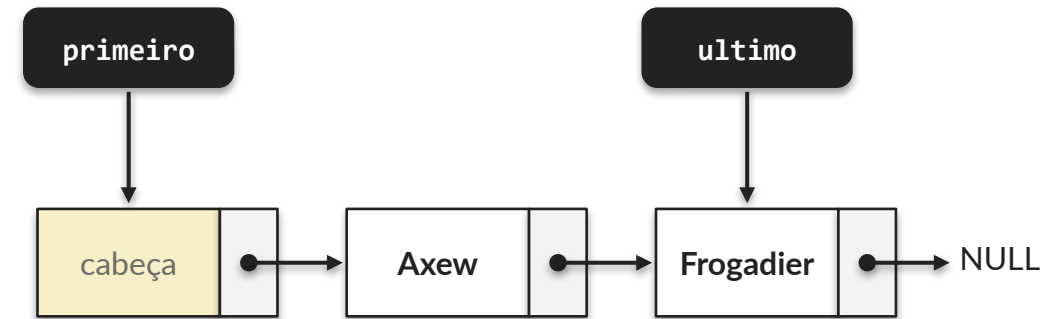
- Os cinco atributos do enunciado; a localização usa o tipo Coordenada.
- O **Id é único** e vem da ordem de leitura. O número da Pokédex pode repetir.
- `pokemonInicializar` preenche tudo chamando os próprios `set`, então nenhum deles fica sem uso.
- Os `set` de nome e tipo usam `strncpy` com `'\0'` explícito: um texto maior que o vetor é cortado, sem estourar.



TAD PokeLista: a estrutura

```
C pokelista.h include X
8 // pokelista: lista encadeada de pokemon
9
10 typedef struct pokecelula {
11     Pokemon pokemon;
12     struct pokecelula *prox;
13 } Pokecelula;
14
15 typedef struct {
16     Pokecelula *primeiro;
17     Pokecelula *ultimo;
18     int tamanho;
19 } Pokelista;
```

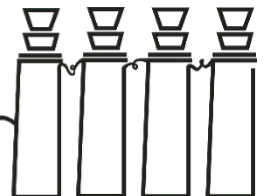
Figura 04 – A célula e a lista, em include/pokelista.h



tamanho = 2

A lista com dois Pokémon

- O Pokémon fica **por valor** na célula: cada lista tem a sua cópia.
- A **célula cabeça** não guarda Pokémon: dá um antecessor ao primeiro, e a lista está vazia quando primeiro == ultimo.
- Operações: inicializar, inserir, remover por Id, remover o primeiro, buscar, imprimir e liberar.



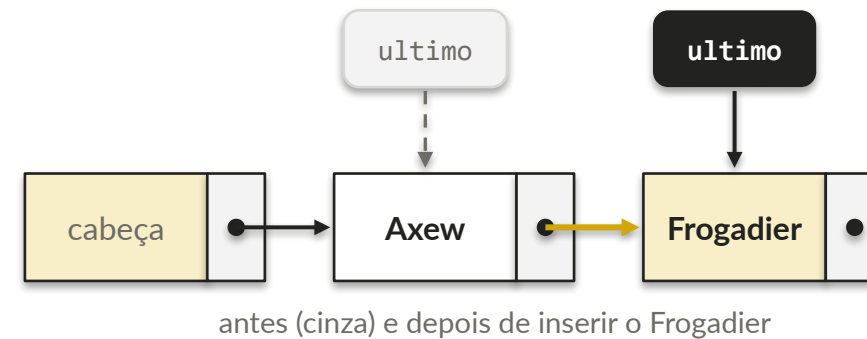
PokeLista: inserção no fim

```

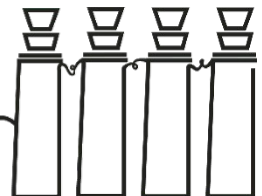
C pokelista.c src X
26 int pokelistaInserir(Pokelista *pl, const Pokemon *p)
27 {
28     Pokecelula *nova;
29
30     nova = (Pokecelula *) malloc(sizeof(Pokecelula));
31     if (nova == NULL) {
32         return 0;
33     }
34
35     // copia a struct inteira: a lista fica dona da sua propria
36     // copia
37     nova->pokemon = *p;
38     nova->prox = NULL;
39     pl->ultimo->prox = nova;
40     pl->ultimo = nova;
41     pl->tamanho++;
42
43     return 1;
44 }

```

Figura 05 – Inserção, em src/pokelista.c



- Aloca a célula nova; se o malloc falhar, devolve 0 e a lista fica como estava.
- Copia a struct inteira (`nova->pokemon = *p`): a lista fica dona da sua cópia.
- Liga a nova depois de ultimo e move ultimo para ela (destaque): **O(1)**, sem percorrer a lista.
- Inserir no fim mantém a ordem do arquivo nos fugitivos e a ordem de captura no treinador.



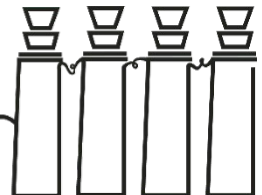
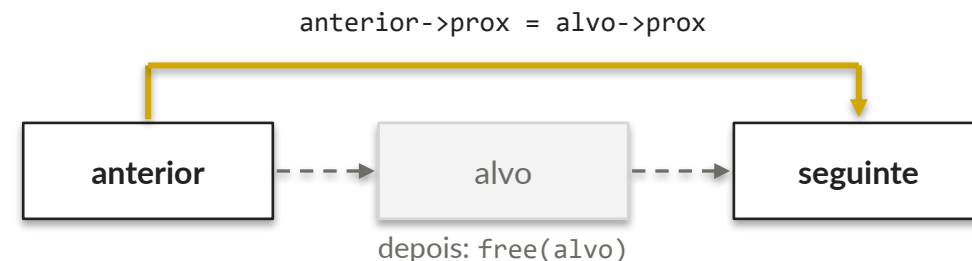
PokeLista: remoção por Id

```

C pokelista.c src X
57 // chegou ao fim sem achar
58 if (anterior->prox == NULL) {
59     return 0;
60 }
61
62 alvo = anterior->prox;
63
64 if (removido != NULL) {
65     *removido = alvo->pokemon;
66 }
67
68 // desliga o alvo da corrente
69 anterior->prox = alvo->prox;
70
71 // sem esta correcao, ultimo ficaria apontando para memoria
72 // liberada e a proxima insercao escreveria nela
73 if (alvo == pl->ultimo) {
74     pl->ultimo = anterior;
75 }
76
77 free(alvo);
78 pl->tamanho--;
79 return 1;
80 }
    
```

Figura O6 – Remoção por Id, em src/pokelista.c

- Antes disso, um laço anda com anterior, a célula **antes** do alvo: numa lista simples não dá para voltar, e é ela quem precisa ser religada.
- O Pokémon é copiado para fora **antes do** free.
- Se o removido era o último, ultimo volta para o anterior (destaque). Sem isso, ele apontaria para memória liberada e a próxima inserção escreveria nela.
- Remover o primeiro reaproveita esta função com o Id do primeiro.



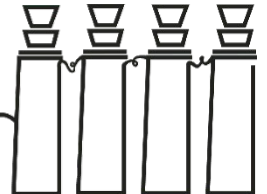
```
C treinador.h include X
15 // a lista guarda os pokemon que ele capturou e ainda nao
    entregou
16 typedef struct {
17     int identificador;
18     char nome[TAM_NOME_COACH];
19     Coordenada loccoach;
20     Pokelista lista;
21     int qntdpokebolas;
22 } Treinador;
```

Figura 07 – A struct, em include/treinador.h

- Começa em (0,0) com a PokeLista vazia. O identificador é a ordem do arquivo (1 e 2) e decide o empate.
- Capturar gasta uma Pokébola **só se a inserção deu certo**; sem Pokébola, devolve 0.
- Retirar tira sempre o primeiro da lista: a entrega sai na ordem de captura.
- Movimentar só muda a posição; as mensagens ficam na missão.

```
C treinador.c src X
65 int treinadorCapturar(Treinador *t, const Pokemon *p)
66 {
67     // sem pokebola nao ha captura
68     if (t->qntdpokebolas <= 0) {
69         return 0;
70     }
71
72     // so gasta a pokebola se a insercao deu certo
73     if (!pokelistaInserir(&t->lista, p)) {
74         return 0;
75     }
76
77     t->qntdpokebolas--;
78
79     return 1;
80 }
81
82 int treinadorRetirarPokemon(Treinador *t, Pokemon *retirado)
83 {
84     // o primeiro da lista e o mais antigo: a entrega sai na
        ordem de captura
85     return pokelistaRemoverPrimeiro(&t->lista, retirado);
86 }
```

Figura 08 – Captura e retirada, em src/treinador.c



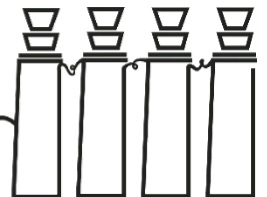
```
C pokecenter.h include X
8 // posicao do centro
9 #define CENTRO_X 0
10 #define CENTRO_Y 0
11
12 // intervalo da recarga de pokebolas
13 #define MIN_RECARGA 1
14 #define MAX_RECARGA 20
15
16 typedef struct {
17     Pokelista fugitivos;
18     Pokelista recuperados;
19     Coordenada locPokeCenter;
20 } PokeCenter;
```

Figura 09 – A struct, em include/pokecenter.h

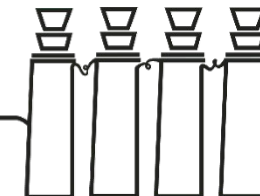
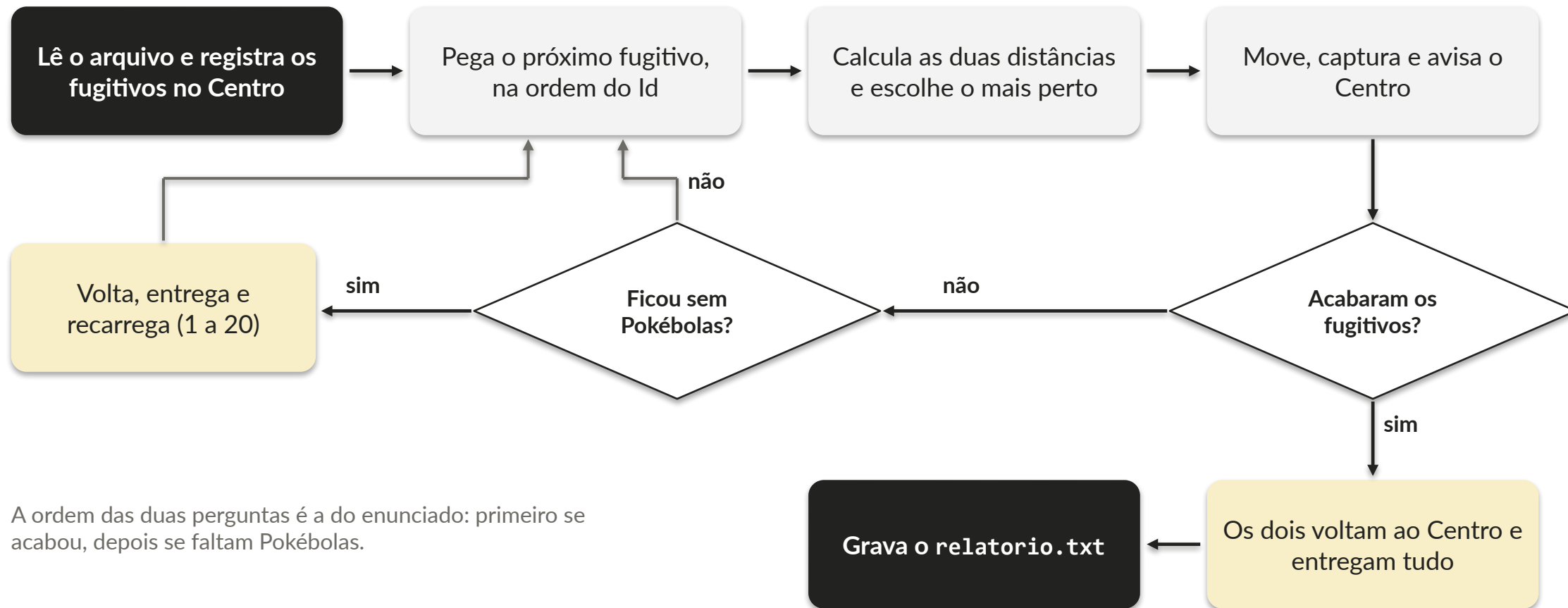
- Duas PokeLista: **fugitivos** (ordem do arquivo) e **recuperados** (ordem de entrega). O Centro fica em (0,0).
- Receber retira um a um os Pokémon do treinador e insere no fim dos recuperados.
- A recarga só vale no Centro e sorteia **de 1 a 20** (destaque). O srand roda uma vez, no main.

```
C pokecenter.c src X
78 int pokecenterRecarregarPokebolas(PokeCenter *cp, Treinador *t)
79 {
80     Coordenada posicaoCentro;
81     Coordenada posicaoTreinador;
82     int quantidade;
83
84     // so recarrega o treinador que esta no centro
85     posicaoCentro = pokecenterGetLocalizacao(cp);
86     posicaoTreinador = treinadorGetLocalizacao(t);
87
88     if (posicaoTreinador.cordX != posicaoCentro.cordX ||
89         posicaoTreinador.cordY != posicaoCentro.cordY) {
90         return 0;
91     }
92
93     // o resto vai de 0 a max - min; somando min, fica entre
94     // min e max
95     quantidade = MIN_RECARGA + rand() % (MAX_RECARGA -
96     MIN_RECARGA + 1);
97
98     treinadorSetPokebolas(t, quantidade);
99
100     return quantidade;
101 }
```

Figura 10 – Recarga de Pokébolas, em src/pokecenter.c



O fluxo da missão



Leitura da entrada

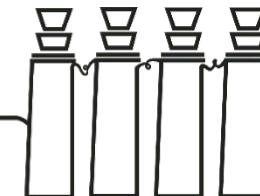
```
C missao.c src X
58 fscanf(entrada, "%d", &total);
59
60 for (i = 0; i < total; i++) {
61     fscanf(entrada, "%d %s %s %d %d", &numPokedex, nome,
62         tipo, &cordX, &cordY);
63
64     // o id e a ordem de leitura. o numero da pokedex nao
65     serviria: o arquivo de teste oficial traz quatro
66     pikachus com o numero 25
67     pokemonInicializar(&p, i + 1, numPokedex, nome, tipo,
68         cordX, cordY);
69
70     if (!pokecenterRegistrarFugitivo(cp, &p)) {
71         printf("Erro: memoria insuficiente para registrar o
72             Pokemon %s.\n", nome);
73         return 0;
74     }
75 }
```

Figura 11 – Leitura dos fugitivos, em src/missao.c

```
teste1.txt testes X
1 Rosa 2
2 Nate 2
3 5
4 610 Axew Dragão 7 7
5 657 Frogadier Água 10 7
6 387 Turtwig Folha 1 1
7 715 Noivern Voador -10 -10
8 197 Umbreon Noturno 5 7
```

Figura 12 – O arquivo testes/teste1.txt

- Antes vêm os dois treinadores (nome e Pokébolas) e a quantidade de Pokémon.
- Cada Pokémon é lido com um único fscanf e ganha **Id = i + 1**.
- As variáveis da leitura começam zeradas: se faltar uma linha, nada fica com lixo de memória.
- O %s para no espaço em branco, então o \r do Windows (CRLF) nunca entra no nome.



A escolha do treinador

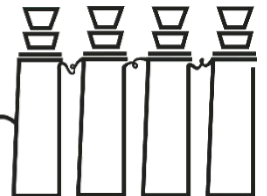
```
missao.c src X
9 float distanciaQuadrado(Coordenada a, Coordenada b)
10 {
11     float dx = (float) a.cordX - b.cordX;
12     float dy = (float) a.cordY - b.cordY;
13
14     return dx * dx + dy * dy;
15 }
16
17 // o mais proximo do alvo e em caso de empate, o de menor
18 // identificador
19 Treinador *escolheTreinador(Treinador *t1, Treinador *t2, float
20 distancia1, float distancia2)
21 {
22     if (distancia1 < distancia2) {
23         return t1;
24     }
25     if (distancia2 < distancia1) {
26         return t2;
27     }
28
29     // empate: a missao vai para o treinador de menor
30     // identificador
31     if (treinadorGetId(t1) <= treinadorGetId(t2)) {
32         return t1;
33     }
34     return t2;
35 }
```

Figura 13 – Distância e escolha, em src/missao.c

- Comparamos o **quadrado** da distância, sem tirar a raiz: a raiz é crescente, então a ordem não muda.
- Em mapas de até uns 2000×2000 , o quadrado sai exato no float, então o empate é detectado sem erro de arredondamento.
- Empate: vai o treinador de **menor identificador**.
- A raiz só aparece na impressão, com `%.2f`.

Exemplo do enunciado

Axew em (7,7), os dois em (0,0): Rosa **9.90** e Nate **9.90**. Empate, a missão vai para a Rosa (Id 1).



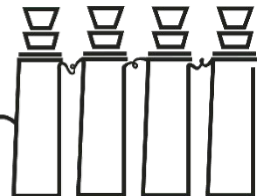
O laço da missão

```

C missao.c src X
219 // os ids sao a ordem de leitura, a busca confirma que o
    pokemon ainda esta fugido antes de montar o resgate
220 for (id = 1; id <= qtdFugitivos; id++) {
221     if (!pokecenterBuscarFugitivo(cp, id, &alvo)) {
222         continue;
223     }
224
225     escolhido = resgataPokemon(cp, t1, t2, &alvo);
226     if (escolhido == NULL) {
227         continue;
228     }
229
230     // a ordem destes dois testes importa: se o pokemon
    capturado era o ultimo, a missao termina sem recarga,
    mesmo que o treinador tenha ficado sem pokebolas
231     if (!pokecenterTemFugitivos(cp)) {
232         break;
233     }
234
235     if (treinadorGetPokebolas(escolhido) == 0) {
236         retornaAoCentro(cp, escolhido);
237     }
238 }
239
240 encerraMissao(cp, t1, t2);
    
```

Figura 14 – O laço, em src/missao.c

- Os Ids são a ordem de leitura; a busca no Centro confirma que o Pokémon ainda está fugido.
- Depois da captura, pergunta primeiro se **acabaram os fugitivos**; só depois se o treinador **ficou sem Pokébolas** (destaque).
- No exemplo do enunciado, o Umbreon zera as Pokébolas da Rosa e a missão termina sem recarga. Com os testes trocados, ela voltaria para buscar Pokébolas que não vai usar.
- Antes do laço, quem começa com zero Pokébolas recarrega.



Os retornos ao Centro

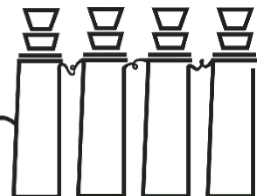
```
C missao.c src X
91 posicaoCentro = pokecenterGetLocalizacao(cp);
92
93 printf("=====\n");
94 printf("      Treinador(a) %s SEM POKEBOLAS  \n",
95       treinadorGetNome(t));
96 printf("=====\n");
97 printf("Treinador(a) %s retorna ao Centro de Pesquisa.\n\n",
98       treinadorGetNome(t));
99 treinadorMovimentar(t, posicaoCentro.cordX, posicaoCentro.cordY);
100 printf("Entregando Pokemon ao Centro de Pesquisa.\n\n");
101 pokecenterReceberPokemon(cp, t);
102
103 recarregaNoCentro(cp, t);
```

Figura 15 – Sem Pokébolas, em src/missao.c

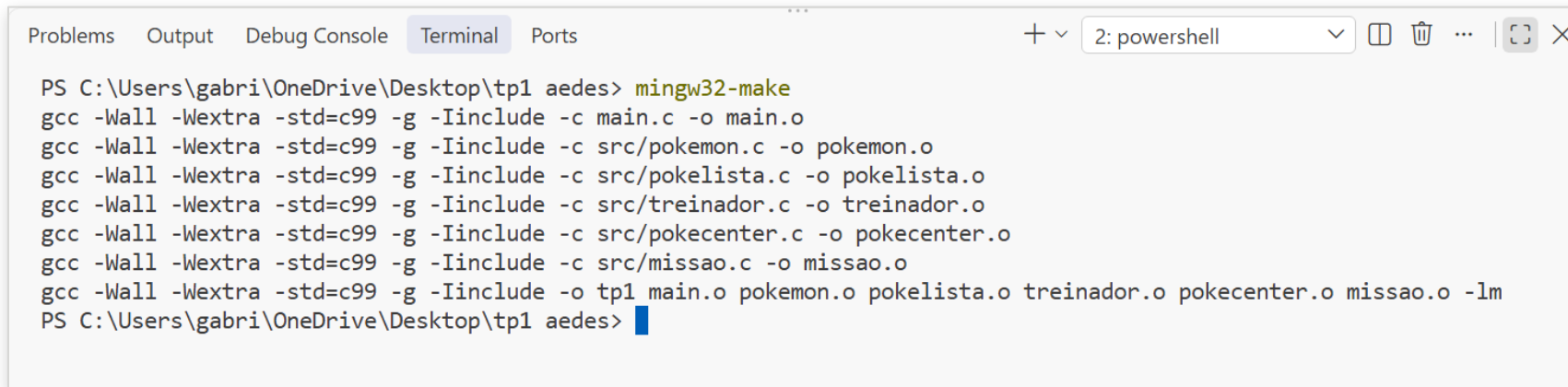
```
C missao.c src X
182 printf("\nAmbos treinadores retornam ao Centro de Pesquisa.\n\n");
183 treinadorMovimentar(t1, posicaoCentro.cordX, posicaoCentro.cordY);
184 treinadorMovimentar(t2, posicaoCentro.cordX, posicaoCentro.cordY);
185
186 // t1 tem o menor identificador, entao entrega primeiro
187 printf("Treinador(a) %s devolve os Pokemon.\n\n",
188       treinadorGetNome(t1));
189 pokecenterReceberPokemon(cp, t1);
190
191 printf("Treinador(a) %s devolve os Pokemon.\n\n",
192       treinadorGetNome(t2));
193 pokecenterReceberPokemon(cp, t2);
```

Figura 16 – Fim da missão, em src/missao.c

- **Sem Pokébolas:** o treinador volta para (0,0), entrega o que capturou e pede um novo carregamento.
- **Último resgate:** os dois voltam, e a Rosa (Id 1) entrega antes do Nate. É essa ordem que faz o relatório sair igual ao do exemplo.
- Entregar é retirar o primeiro da lista do treinador e inserir no fim dos recuperados: a ordem de captura se mantém.



Execução: compilação e entrada

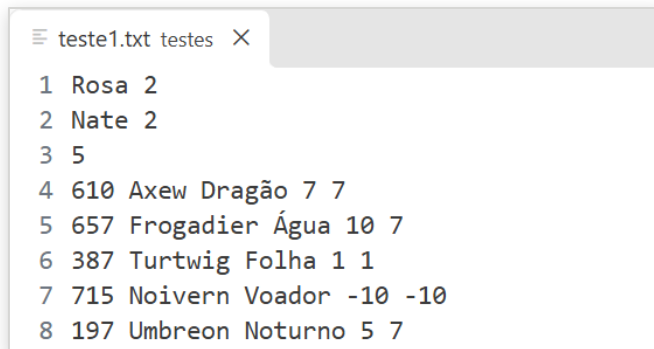


```

PS C:\Users\gabri\OneDrive\Desktop\tp1 aedes> mingw32-make
gcc -Wall -Wextra -std=c99 -g -Iinclude -c main.c -o main.o
gcc -Wall -Wextra -std=c99 -g -Iinclude -c src/pokemon.c -o pokemon.o
gcc -Wall -Wextra -std=c99 -g -Iinclude -c src/pokelista.c -o pokelista.o
gcc -Wall -Wextra -std=c99 -g -Iinclude -c src/treinador.c -o treinador.o
gcc -Wall -Wextra -std=c99 -g -Iinclude -c src/pokecenter.c -o pokecenter.o
gcc -Wall -Wextra -std=c99 -g -Iinclude -c src/missao.c -o missao.o
gcc -Wall -Wextra -std=c99 -g -Iinclude -o tp1 main.o pokemon.o pokelista.o treinador.o pokecenter.o missao.o -lm
PS C:\Users\gabri\OneDrive\Desktop\tp1 aedes>

```

Figura 17 – Compilação com o Makefile, no terminal do VS Code



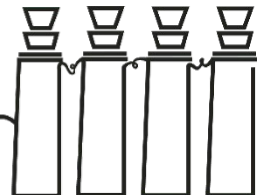
```

1 Rosa 2
2 Nate 2
3 5
4 610 Axew Dragão 7 7
5 657 Frogadier Água 10 7
6 387 Turtwig Folha 1 1
7 715 Noivern Voador -10 -10
8 197 Umbreon Noturno 5 7

```

Figura 18 – O arquivo de entrada

- mingw32-make compila os seis arquivos com -Wall -Wextra -std=c99, **sem nenhum aviso.**
- O programa pede o caminho do arquivo de entrada.
- Formato: os dois treinadores (nome e Pokébolas), a quantidade de Pokémon e uma linha por Pokémon: Pokédex, nome, tipo, X e Y.



Execução: resgates e retorno

```
Problems Output Debug Console Terminal Ports

PS C:\Users\gabri\OneDrive\Desktop\tp1 aedes> .\tp1.exe
Caminho do arquivo de entrada: testes\teste1.txt
=====
                INICIO DA MISSAO
=====

Treinador(a) Rosa: posicao (0,0) | Pokebolas: 2
Treinador(a) Nate: posicao (0,0) | Pokebolas: 2

Pokemons fugitivos a serem resgatados: 5

-----

Pokemon alvo: Axew
Localizacao: (7,7)

Distancia Treinador(a) Rosa: 9.90
Distancia Treinador(a) Nate: 9.90

Missao atribuida ao Treinador(a) Rosa.

Treinador(a) Rosa se movimentou para (7,7).
Axew capturado com sucesso!

Pokebolas restantes para o Treinador(a) Rosa: 1
```

Figura 19 – O início: no empate de 9.90, vai a Rosa (Id 1)

```
Problems Output Debug Console Terminal Ports

Pokebolas restantes para o Treinador(a) Rosa: 0

=====
                Treinador(a) Rosa SEM POKEBOLAS
=====
Treinador(a) Rosa retorna ao Centro de Pesquisa.

Entregando Pokemon ao Centro de Pesquisa.

Treinador(a) Rosa recebeu 18 Pokebolas.

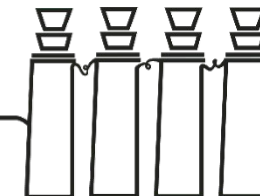
-----

Pokemon alvo: Turtwig
Localizacao: (1,1)

Distancia Treinador(a) Rosa: 1.41
Distancia Treinador(a) Nate: 1.41
```

Figura 20 – A Rosa fica sem Pokébolos, volta e recarrega

- Nesta execução o sorteio deu **18** Pokébolos.
- A distância **1.41** até o Turtwig mostra que ela voltou para (0,0).



Execução: fim e relatório

```

Problems  Output  Debug Console  Terminal  Ports

Pokemon alvo: Umbreon
Localizacao: (5,7)

Distancia Treinador(a) Rosa: 7.21
Distancia Treinador(a) Nate: 22.67

Missao atribuida ao Treinador(a) Rosa.

Treinador(a) Rosa se movimentou para (5,7).
Umbreon capturado com sucesso!

Pokebolas restantes para o Treinador(a) Rosa: 16

=====
      Todos Pokemons foram resgatados
=====

Ambos treinadores retornam ao Centro de Pesquisa.

Treinador(a) Rosa devolve os Pokemon.

Treinador(a) Nate devolve os Pokemon.

=====
      MISSAO CONCLUIDA
=====

PS C:\Users\gabri\OneDrive\Desktop\tp1 aedes>

```

Figura 21 – O fim da missão

```

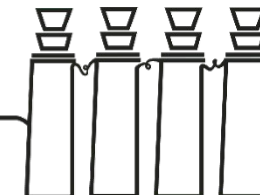
relatorio.txt tp1 aedes X

1 Pokemon recuperados:
2 610 Axew
3 657 Frogadier
4 387 Turtwig
5 197 Umbreon
6 715 Noivern

```

Figura 22 – O relatorio.txt gravado

- O Umbreon é o último: a missão termina sem passar pela recarga.
- Os dois voltam e devolvem os Pokémon, a Rosa primeiro.
- O relatório sai na ordem de entrega: 610 e 657 (primeira volta da Rosa), 387 e 197 (fim) e 715 (Nate), **igual ao exemplo do enunciado**.
- O sorteio muda as Pokébolas de cada execução; o fluxo e o relatório, não.



34

arquivos de entrada, todos
até o fim

10 / 10

distâncias do exemplo do
enunciado

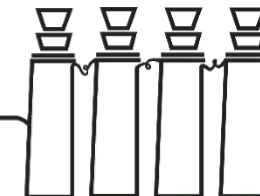
0

avisos com -Wall -Wextra

64 = 64

malloc e free no teste2

- teste1.txt: a saída é a do enunciado, a menos dos espaços das molduras, e o relatório sai idêntico.
- teste2.txt: 20 resgates; os quatro Pikachus 025 são Pokémon diferentes, porque o Id é a ordem de leitura.
- Casos nossos: empate na origem, zero e uma Pokébola, coordenadas negativas, arquivo em CRLF, arquivo vazio, nomes longos e 500 Pokémon. Nenhum trava.
- Memória: 4 células cabeça + 3 por Pokémon, e cada malloc tem o seu free (500 Pokémon: 1504 e 1504).



O que não fechava no enunciado

O relatório

O texto pede "o ID e o Nome", mas o exemplo imprime 610 Axew: 610 é o número da Pokédex.

Decisão: Seguimos o exemplo.

O arquivo de entrada

No PDF o bloco está em três colunas e, copiado, junta 7 7 em 77.

Decisão: Conferimos com o arquivo oficial.

O Id do Pokémon

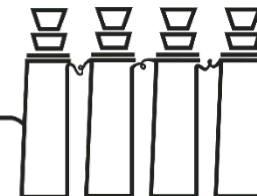
O Id precisa ser único, mas o arquivo de entrada não traz Id.

Decisão: O Id é a ordem de leitura.

O espaço no relatório

No exemplo, cada linha do relatório começa com um espaço, que é recuo do documento.

Decisão: Gravamos sem o espaço.



1

Uma lista para tudo

A mesma PokeLista, com célula cabeça e apontador ultimo, serve aos fugitivos, aos recuperados e aos dois treinadores.

2

TADs separados

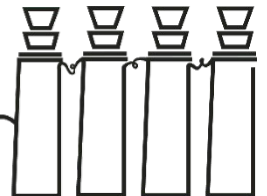
Cada TAD tem a sua struct e as suas operações, e a missão só usa essas operações: cada parte pode ser testada e explicada sozinha.

3

Testado de verdade

Os arquivos oficiais e os nossos casos confirmam o fluxo do enunciado, sem travar e sem vazar memória.

Obrigado! Perguntas?



INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 9899:1999**: programming languages — C. Genebra: ISO, 1999.

KERNIGHAN, B. W.; RITCHIE, D. M. **C: a linguagem de programação**. 2. ed. Rio de Janeiro: Campus, 1989.

SILVA, T. R. M. B. **Trabalho Prático 1**: Algoritmos e Estruturas de Dados 1. Florestal: Universidade Federal de Viçosa, 2026. Especificação do trabalho.

ZIVIANI, N. **Projeto de algoritmos**: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.

